

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	j.venkata varun	Reg. No.:	192372227
Section / Batch:	AI & DS	Date:	15-07-26

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. E-Commerce Recommendation System

An e-commerce system uses two algorithms with complexities $O(n \log n)$ and $O(n^2)$. Analyze both using asymptotic notation. Compare their performance for large datasets. Explain growth rate differences and justify which is more efficient. Provide a conclusion based on scalability.

Q2. Cloud Storage Processing System

A cloud system processes files using $T(n) = 3T(n/2) + n$. Apply the Master Theorem to find the time complexity. Identify parameters and justify the case selection. Analyze how this impacts performance for large storage systems. Interpret efficiency using asymptotic notation.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

E-Commerce Recommendation System

Introduction

An e-commerce recommendation system uses two algorithms with time complexities $O(n \log n)$ and $O(n^2)$. Asymptotic notation is used to analyze how the running time of these algorithms grows as the input size (n) increases.

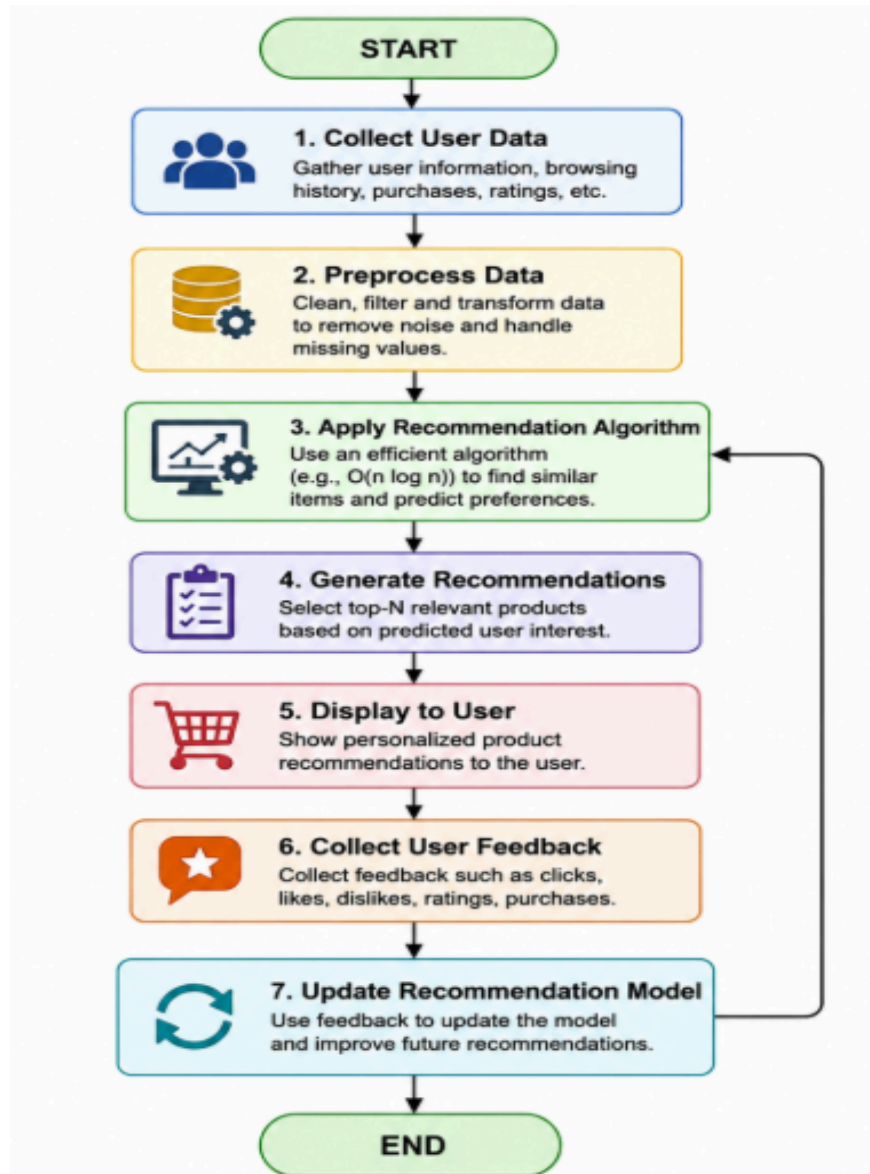
1. Analysis Using Asymptotic Notation

Algorithm 1: $O(n \log n)$

- Big-O Notation: $O(n \log n)$
- This algorithm's execution time increases proportionally to $n \times \log n$.
- It is more efficient than quadratic algorithms because the logarithmic factor grows very slowly.
- Common examples include Merge Sort, Heap Sort, and efficient searching or recommendation algorithms.

Algorithm 2: $O(n^2)$

- Big-O Notation: $O(n^2)$
- This algorithm's execution time increases proportionally to the square of the input size.
- As the dataset becomes larger, the running time grows very rapidly.
- Common examples include Bubble Sort, Selection Sort, and algorithms using nested loops.



2. Performance Comparison for Large Datasets

Input Size (n)	$O(n \log n)$	$O(n^2)$
100	≈ 664 operations	10,000 operations
1,000	≈ 9,966 operations	1,000,000 operations
10,000	≈ 132,877 operations	100,000,000 operations

Code:

```
import matplotlib.pyplot as plt
import math
n = [10, 100, 1000, 10000, 100000]
n_log_n = [x * math.log2(x) for x in n]
n_square = [x ** 2 for x in n]
plt.figure(figsize=(8,5))
plt.plot(n, n_log_n, marker='o', label='O(n log n)')
plt.plot(n, n_square, marker='s', label='O(n²)')
plt.xscale('log')
plt.yscale('log')
plt.title("Comparison of O(n log n) and O(n²)")
plt.xlabel("Input Size (n)")
plt.ylabel("Operations")
plt.legend()
plt.grid(True)
plt.show()
```

Observation:

- For small datasets, both algorithms may perform reasonably well.
- For large datasets, $O(n^2)$ requires significantly more operations than $O(n \log n)$.
- Therefore, $O(n \log n)$ performs much faster and uses computational resources more efficiently.

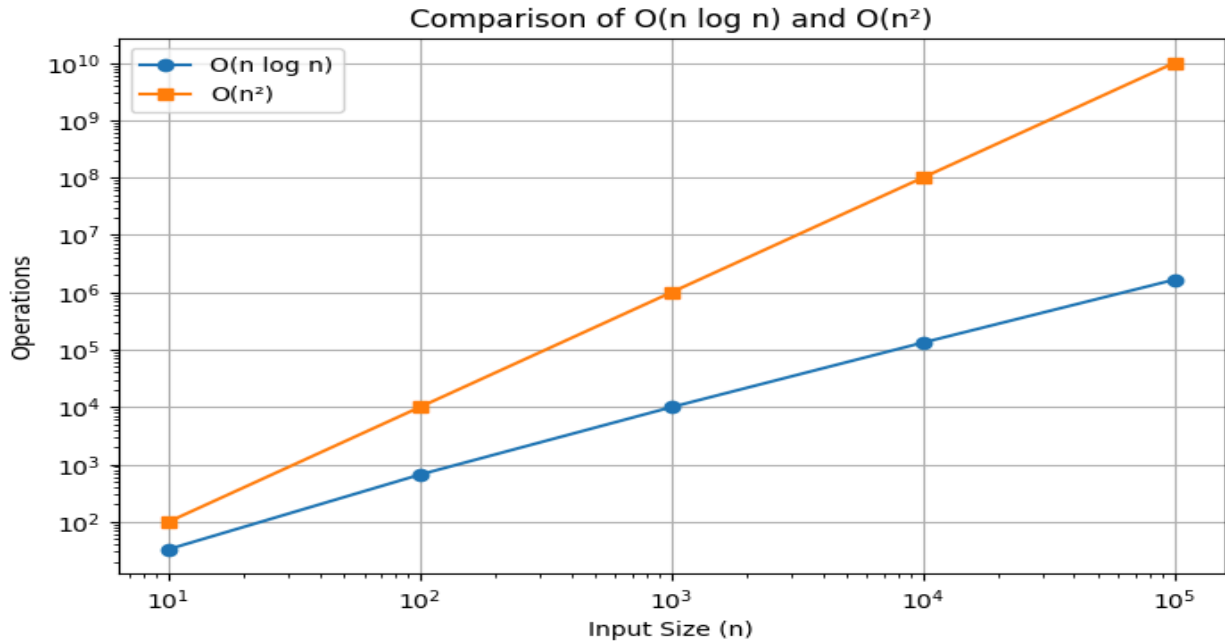
3. Growth Rate Differences

$O(n \log n)$

- Growth rate is near-linear.
- The logarithmic factor ($\log n$) increases slowly.
- Execution time increases gradually even as the dataset grows.
- Suitable for handling large-scale e-commerce data.

$O(n^2)$

- Growth rate is quadratic.
- Doubling the input size increases the running time by approximately four times.
- Performance decreases rapidly for large datasets.
- Not suitable for systems with millions of products or users.



4. Which Algorithm is More Efficient?

The $O(n \log n)$ algorithm is more efficient because:

- It requires far fewer operations than $O(n^2)$.
- It processes large datasets faster.
- It consumes less processing time and computational resources.
- It provides better performance for real-time recommendation systems where quick responses are essential.

The $O(n^2)$ algorithm becomes inefficient as the number of products or users increases because its running time grows much more rapidly.

5. Conclusion

Based on asymptotic analysis, the $O(n \log n)$ algorithm is the better choice for an e-commerce recommendation system. Its slower growth rate allows it to handle large datasets efficiently, making it highly scalable and suitable for real-world applications. In contrast, the $O(n^2)$ algorithm does not scale well because its execution time increases rapidly as the input size grows. Therefore, $O(n \log n)$ is the preferred algorithm for achieving faster performance, better scalability, and efficient handling of large e-commerce datasets.

Cloud Storage Processing System

1. Given Recurrence Relation

The cloud storage processing system follows the recurrence:

$$T(n)=3T(n/2)+n$$

This recurrence can be solved using the Master Theorem.

2. Identify the Parameters

The standard form of the Master Theorem is:

$$T(n)=aT(n/b)+f(n)$$

Comparing with the given recurrence:

- $a = 3$
- $b = 2$
- $f(n) = n$

Now calculate:

$$n^{\log_b a} = n^{\log_2 3}$$

Since

$$\log_2 3 \approx 1.585$$

Therefore,

$$n^{\log_2 3} = n^{1.585}$$

3. Apply the Master Theorem

Compare:

- $f(n)=n$
- $n^{\log_2 3}=n^{1.585}$

Since

$$n < n^{1.585}$$

Or

$$f(n) = O(n^{\log_2 3 - \epsilon})$$

for some $\epsilon > 0$, this satisfies Master Theorem Case 1.

4. Case Selection Justification

Master Theorem Case 1

If

$$f(n) = O(n^{\log_b a - \epsilon})$$

Then

$$T(n) = \Theta(n^{\log_b a})$$

Since

- $a=3$
- $b=2$
- $f(n)=n$

And

$$n = O(n^{1.585 - \epsilon})$$

the recurrence belongs to Case 1.

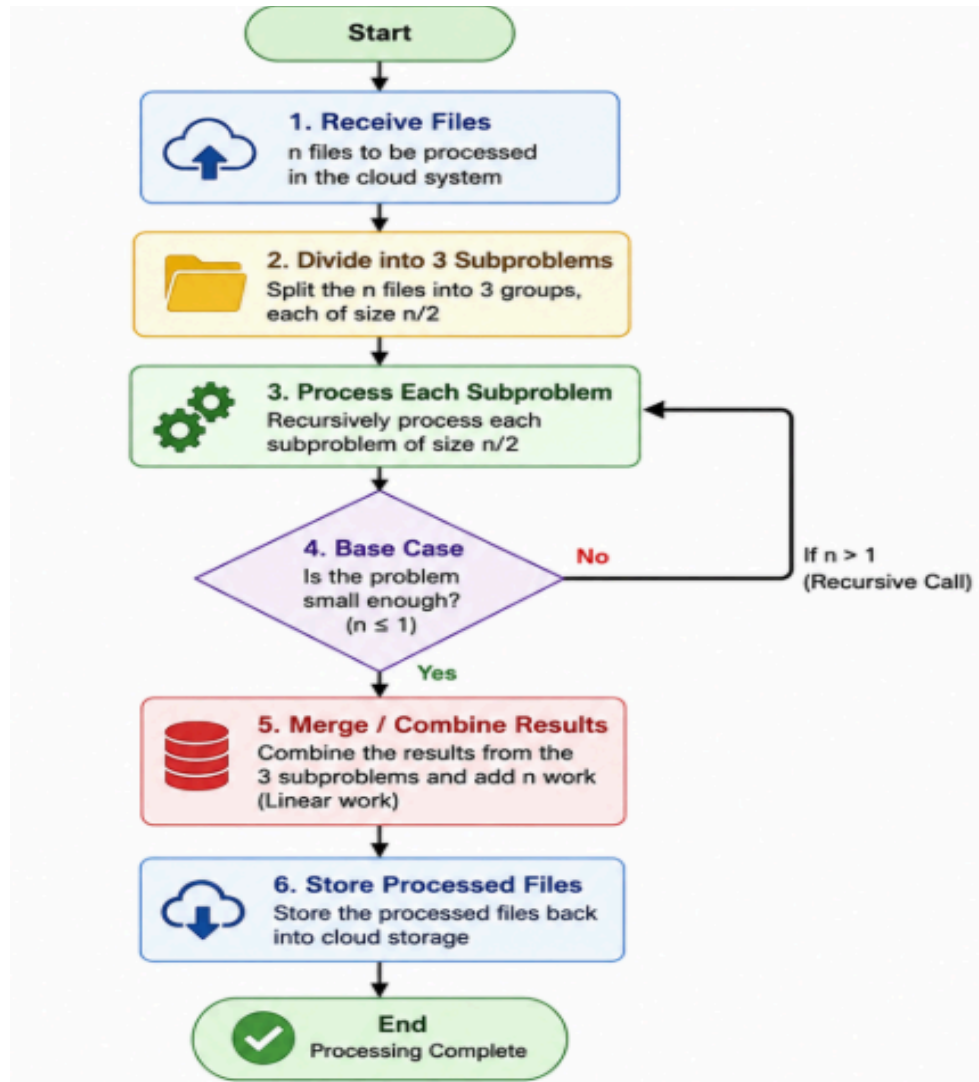
5. Final Time Complexity

Therefore,

$$T(n) = \Theta(n^{\log_2 3})$$

or approximately,

$$T(n) = \Theta(n^{1.585})$$



6. Performance Analysis for Large Storage Systems

Input Size (n)	Approximate Growth ($n^{\{1.585\}}$)
100	$\approx 1,482$
1,000	$\approx 57,544$
10,000	$\approx 2,236,068$
100,000	$\approx 86,975,000$

Observation

- The running time grows faster than $O(n)$.
- It grows much slower than $O(n^2)$.
- As the number of cloud files increases, processing time increases moderately.
- The algorithm remains efficient for handling large cloud storage systems.

7. Efficiency Using Asymptotic Notation

- **Big-O:** $O(n^{1.585})$
- **Big-Theta:** $\Theta(n^{1.585})$
- **Big-Omega:** $\Omega(n^{1.585})$

This means the algorithm's running time is tightly bounded by $n^{1.585}$.

8. Python Code

```
import math
n_values = [100, 1000, 10000, 100000]
print("n\tT(n) ≈ n^(log2 3)")
for n in n_values:
    operations = n ** (math.log2(3))
    print(f"{n}\t{operations:.2f}")
```

```
import math
n_values = [100, 1000, 10000, 100000]
print("n\tT(n) ≈ n^(log2 3)")
for n in n_values:
    operations = n ** (math.log2(3))
    print(f"{n}\t{operations:.2f}")
```

n	T(n) ≈ n^(log2 3)
100	1478.85
1000	56870.56
10000	2187006.14
100000	84103196.75

Conclusion

Using the Master Theorem, the recurrence relation

$$T(n)=3T(n/2)+n$$

belongs to Case 1, where $a = 3$, $b = 2$, and $f(n) = n$. The resulting time complexity is

$$\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

This complexity grows faster than linear time but much slower than quadratic time, making it suitable for large cloud storage systems. As the number of files increases, the processing time remains scalable and efficient, ensuring good performance for large-scale cloud applications.